
CAS2105 Homework 6: Mini AI Pipeline Project 🙌

Sookyeong Kim (2022148076)

1 Introduction

This project provides a gentle introduction to designing simple **AI pipelines**. Rather than training large models or reading extensive research literature, you will:

- Choose a small, concrete problem solvable with an AI pipeline (e.g., text classification, retrieval, simple QA, image classification).
- Choose or collect a small dataset (e.g., from **datasets**).
- Implement a simple *naïve baseline* (e.g., rule-based or heuristic).
- Build an improved pipeline using existing pre-trained models.
- Evaluate both approaches using appropriate metrics.
- Reflect on what worked, what failed, and what you learned.

The emphasis is on the *process* of AI work: problem definition, pipeline design, evaluation, iteration, and writing. Your problem should be small enough to run comfortably on a single GPU (e.g., RTX 3090) or CPU.

Your tasks. Please replace all the sections to complete your report, starting from adding your project title, your name, and student ID above! Feel free to reorganize the sections. Then, submit a **public** github repository link that includes your code (including Jupyter notebook with results) and report **in PDF**, via LearnUs.

You can use tables and figures, and cite references using `\citep` ([1]) or `\citet` (Wei et al. [1]).

2 Task Definition

- **Task description:** What are you trying to do? (e.g., “Classify news headlines into {sports, politics, tech}”).
- **Motivation:** Why is this task interesting or useful?
- **Input / Output:** What does the model see, and what should it produce?
- **Success criteria:** How do you know if your system is “good”?

3 Methods

This section includes both the naïve baseline and the improved AI pipeline.

3.1 Naïve Baseline

Implement a simple method that does not rely on heavy models. Examples include:

- Keyword-based text classification,
- Simple color/shape heuristics for image tasks,
- String-overlap-based retrieval.

In your report, explain:

- How the baseline works,
- Why it is considered naïve,
- Expected failure cases.

Your Baseline (TODO)

- **Method description:** The baseline classifier uses a rule-based keyword matching approach. For each news headline, the text is lowercased and checked against predefined keyword lists corresponding to each category: Sports, Business, and Sci/Tech. If a headline contains any keyword from a category, it is assigned to that category. If multiple categories match, the first matching rule is applied. Headlines that do not match any keywords are classified as World by default.
- **Why naïve:** This approach is considered naïve because it relies solely on surface-level keyword presence and does not model semantic meaning, context, or word relationships. It cannot generalize beyond the manually defined rules and requires human intuition to select keywords. The method does not learn from data and cannot adapt to new expressions or vocabulary.
- **Likely failure modes:** The baseline is likely to fail in cases where:
 - Headlines use ambiguous words (e.g., “Apple” referring to a company vs. a fruit).
 - Important context is implied rather than explicitly stated through keywords.
 - Synonyms or paraphrases are used that are not included in the keyword lists.
 - Headlines contain keywords from multiple categories, leading to incorrect rule-based decisions.
 - Non-English terms, abbreviations, or creative wording appear.

3.2 AI Pipeline

Design a small pipeline using one or more pre-trained models. Examples include:

- **Text:** embedding encoder + classifier, or a small transformer model,
- **Retrieval:** embedding model + nearest-neighbor search,
- **Vision:** pre-trained classifier (e.g., ViT-tiny).

A typical pipeline contains:

1. Preprocessing,
2. Embedding or representation,
3. Decision/ranking component,
4. Optional post-processing.

Fine-tuning large models is not required; inference-only usage is sufficient.

Your Pipeline (TODO)

- **Models used:** The pipeline uses DistilBERT (distilbert-base-uncased) as a pre-trained transformer encoder for text classification. DistilBERT is a lightweight version of BERT that retains most of its performance while being faster and more computationally efficient. A classification head is added on top of the encoder to predict one of the four news categories: World, Sports, Business, and Sci/Tech.
- **Pipeline stages:**
 1. Preprocessing
News headlines are lowercased and tokenized using the DistilBERT tokenizer. Inputs are truncated or padded to a maximum length of 128 tokens to ensure consistent input size.
 2. Representation (Encoding)
Tokenized headlines are passed through the DistilBERT encoder, which produces contextualized embeddings that capture semantic and syntactic information.
 3. Decision (Classification)
The pooled output embedding is fed into a linear classification head that outputs logits for the four classes. The class with the highest probability is selected as the final prediction.
 4. Post-processing
Predicted class indices are mapped to human-readable category labels for evaluation and comparison.
- **Design choices and justification:** DistilBERT was chosen because it offers a strong balance between performance and efficiency, making it suitable for a small-scale project with limited computational resources. Using a pre-trained transformer allows the model to leverage linguistic knowledge learned from large corpora, which significantly improves generalization compared to rule-based methods. The pipeline is intentionally kept simple to focus on clarity and reproducibility rather than architectural complexity. A maximum sequence length of 128 tokens was selected because news headlines are short, reducing unnecessary computation while preserving relevant information.

4 Experiments

4.1 Datasets

You may use a small public dataset (e.g., from `datasets`) or construct your own. In this section, describe:

- **Dataset source:** where it comes from.
- **Size:** number of examples used.
- **Splits:** how you divided train/validation/test.
- **Preprocessing:** e.g., tokenization, resizing, truncation, normalization.

Your Dataset Description (TODO)

- **Source:** The dataset is a subset of the AG News dataset obtained from the Hugging Face datasets library. The AG News dataset contains news headlines labeled into four categories: World, Sports, Business, and Sci/Tech.
- **Total examples:** A total of 2,500 news headlines were used in this project.
- **Train/Test split:** The dataset was split into 2,000 training examples and 500 test examples. The split was performed randomly while preserving the original label distribution.

- **Preprocessing steps:** All headlines were lowercased and tokenized using the DistilBERT tokenizer. Inputs were truncated or padded to a maximum sequence length of 128 tokens. No additional data augmentation was applied.

4.2 Metrics

Use at least one quantitative metric appropriate for your task:

- **Classification:** accuracy, precision, recall, F1,
- **Retrieval:** precision@k, recall@k,
- **Simple generation:** exact match, ROUGE-1.

It's worth considering how the metrics you select align with your tasks.

4.3 Results

Report:

- Metric values for baseline vs. pipeline,
- A results table,
- At least three qualitative examples.

Method	Accuracy	F1
Baseline	0.380	0.403
AI Pipeline	0.878	0.879

If you want to visualize results with any other than tables, refer to below links

- [Matplotlib official tutorial: Introduction to pyplot](#)
- [Matplotlib example gallery \(many bar/line/scatter plots with source code\)](#)
- [Kaggle notebook: Matplotlib tutorial for beginners \(interactive code\)](#)

5 Reflection and Limitations

Write approximately 6–10 sentences reflecting on:

- What worked better than expected,
- What failed or was difficult,
- How well your metric captured “quality”,
- What you would try next with more time or compute.

Your Reflection (TODO)

The keyword-based baseline performed worse than expected on ambiguous headlines, especially when words like “Apple” or “Amazon” appeared outside their usual contexts. The Transformer-based model significantly improved performance by capturing semantic meaning rather than relying on surface-level keywords.

However, training required more computational resources and careful tokenization. Accuracy and F1-score were useful metrics, but they did not fully reflect model confidence or uncertainty. One limitation of this project is the small dataset size, which may limit generalization. Given more time, I would experiment with data augmentation or compare multiple pre-trained models.

References

- [1] Jason Wei, Maarten Bosma, Vincent Zhao, Kelvin Guu, Adams Wei Yu, Brian Lester, Nan Du, Andrew M. Dai, and Quoc V Le. Finetuned language models are zero-shot learners. In *International Conference on Learning Representations (ICLR)*, 2022. URL <https://openreview.net/forum?id=gEZrGCozdqR>.